📌 JavaScript Events

As a JavaScript developer, understanding **Events** is critical because JavaScript is fundamentally **event-driven**.

What is an Event?

An Event is an action or occurrence that happens in the browser which JavaScript can respond to.

Simple Definition

An event is a signal that something has happened in the browser.

Events can be triggered by:

- 👤 **User actions (click, type, scroll)**
- 🌐 **Browser actions (page load, resize)**
- 📡 **System actions (network response, error)**

How JavaScript Works with Events

When an event occurs:

1. Browser detects the event
2. Browser creates an Event Object
3. JavaScript executes a function (Event Handler)

This is called Event Handling.

What is Event Handling?

👉 **Event Handling means executing a function when an event occurs.**

Syntax:

```javascript
element.addEventListener("eventName", function() {
    // code to execute
});
```

Types of Events in JavaScript

Mouse Events

Event	Description
click	When element is clicked
dblclick	Double click
mousedown	Mouse button pressed
mouseup	Mouse button released
mouseover	Mouse enters element
mouseout	Mouse leaves element

Demo – Click Event

```
HTML

<button id="btn">Click Me</button>

<script>
document.getElementById("btn").addEventListener("click", function() {
    alert("Button Clicked!");
});
</script>
```

Keyboard Events

Event	Description
keydown	Key is pressed
keyup	Key is released
keypress	Key pressed (deprecated)

Demo – Detect Key Press

```
<input type="text" id="inputBox" placeholder="Type something">

<script>
document.getElementById("inputBox").addEventListener("keydown", function(event)
    console.log("Key Pressed: " + event.key);
});
</script>
```

👉 `event.key` gives the key pressed.

Form Events

Event	Description
submit	Form submitted
change	Input value changed
focus	Element gets focus
blur	Element loses focus

Demo – Prevent Form Submission

```
<form id="myForm">
    <input type="text" id="name">
    <button type="submit">Submit</button>
</form>

<script>
document.getElementById("myForm").addEventListener("submit", function(event) {
    event.preventDefault(); // stops page reload
    alert("Form Submitted!");
});
</script>
```

Window / Browser Events

Event	Description
load	Page loaded
resize	Window resized
scroll	Page scrolled
unload	Page closing

```html
<script>
window.addEventListener("load", function() {
    console.log("Page Fully Loaded");
});
</script>
```

Ways to Handle Events

1. Inline Event Handling (Not Recommended)

```html
<button onclick="alert('Clicked!')">Click</button>
```

2. Using DOM Property

```html
<button id="btn">Click</button>

<script>
document.getElementById("btn").onclick = function() {
    alert("Clicked using property!");
};
</script>
```

⚠ Problem: Only one handler allowed.

3. Using addEventListener() (Best Practice ✅)

```html
HTML

<button id="btn">Click</button>

<script>
document.getElementById("btn").addEventListener("click", function() {
    alert("Best Practice Method");
});
</script>
```

✓ Allows multiple event listeners
✓ Cleaner code
✓ Separation of concerns

The Event Object

Whenever an event occurs, JavaScript creates an **Event Object**.

```javascript
JavaScript

document.getElementById("btn").addEventListener("click", function(event) {
    console.log(event);
});
```

Common properties:

Property	**Description**
event.target	Element that triggered event
event.type	Type of event
event.key	Pressed key
event.preventDefault()	Stop default behavior
event.stopPropagation()	Stop bubbling

Event Propagation (Advanced Concept)

What Is Event Bubbling in JavaScript?

Event Bubbling is the mechanism where an event starts from the **target element** (the element that triggered the event) and then propagates upward through its ancestor elements in the DOM hierarchy.

In simple terms:

When you click a child element, the event first runs on that element, then on its parent, then on the grandparent, and continues upward until it reaches the root.

Why Does Event Bubbling Exist?

Because the browser implements an **event propagation model**, which has three phases:

1. **Capturing phase** (top → down)
2. **Target phase**
3. **Bubbling phase** (bottom → up)

By default, JavaScript event listeners run during the **bubbling phase**.

Demo – Bubbling

```
<div id="parent">
    <button id="child">Click</button>
</div>

<script>
document.getElementById("parent").addEventListener("click", function() {
    alert("Parent Clicked");
});

document.getElementById("child").addEventListener("click", function() {
    alert("Child Clicked");
});
</script>
```

👉 Clicking button shows:

Child Clicked
Parent Clicked

Because of **event bubbling**.

What Is Event Delegation in JavaScript?

Event Delegation is a technique where instead of attaching event listeners to multiple child elements, we attach a **single event listener to their parent** and use **event bubbling** to handle events for the children.

🎯 **Simple Definition**

Event delegation is handling events at a parent level for child elements using event bubbling.

🧠 **Why Do We Need Event Delegation?**

Imagine you have **100 buttons inside a list**.

❌ Bad Approach:

```
document.querySelectorAll("button").forEach(btn => {
  btn.addEventListener("click", handler);
});
```

- 100 event listeners
- Poor performance
- Hard to manage
- Won’t work for dynamically added elements

✅ Good Approach (Event Delegation):

```
parent.addEventListener("click", function(event) {
  if(event.target.tagName === "BUTTON") {
    handler();
  }
});
```

Only **ONE listener** handles everything.